॥ श्रीगणेशाय नमः ॥

Uninformed Search



Search

- Exploring a space of possible solutions to find the most suitable/optimal one for a given problem.
- The agents that help us to solve problems are called problem-solving agents.
- E.g. a maze in which we start from a point (start) and end at another (goal).

Problem-solving process

1. Goal formulation:
- The agent adopts the goal to reach B in the maze.

2. Problem formulation:
- Finalizing the states and actions necessary to reach the goal.
- In the above example, an action can be moving from a node to any of its adjacent ones.

3. Search:
- This will be the process prior to the action.
- Simulate the sequence of actions in the model and find the one that reaches the goal.
- That path is called a solution.

4. Execution:
- Executing the actions in the solution, one by one.

Deterministic and non-deterministic environments

Deterministic environments:
- Predictability: The outcome of an action is predefined and predictable.
- Certainty: Next state can be determined with certainty.
- Reproducibility: Performing the same action in the same state will always lead to the same result.

E.g. Playing chess: Given the state of the board and a player's move, the result will be the same.

Non-deterministic environments:
- Unpredictability
- Uncertainty
- Not reproducible

E.g. Driving a car in traffic.

Open loop and closed loop

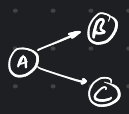
Open loop:
- In deterministic environments, the solution to any problem is a fixed sequence of actions. A→1, 1→2, 2→4
- In a correct model, if the agent has the solution, then it can ignore its percepts while executing the action sequence.

Closed loop:
- This is associated with non-deterministic environments.
- Solution will be a branching strategy (dynamic), where actions depend on percepts.

Formally defining a search problem

State space:
- Set of the sequences of all possible states.

Actions:
- Given a state S, Actions(S): finite set of actions that can be performed from S.
- E.g. Actions(A) = {to B, to C}



Transition model:
- It describes what each action does.
  Result(s, a) → returns the state that results from performing action 'a' on state 's'.
- Eg. Result(A, to B) → B.

Action cost function:
- Numeric cost of applying action a in a state s to reach s'.
  cost(s, a, s')
- Used as a performance measure.
- E.g. for shortest path finding agents, cost = distance between 2 nodes/states.

Consider a 2-cell vacuum world

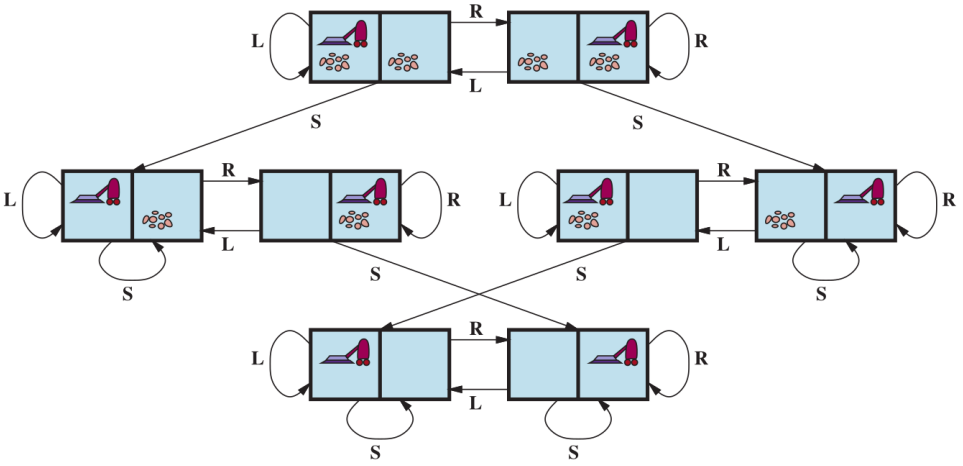


Figure 3.2 The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = *Left*, R = *Right*, S = *Suck*.

Consider the 8-puzzle problem.

Initial state :

3	7	2
5		1
6	4	8

Final state :

	1	2
3	4	5
6	7	8

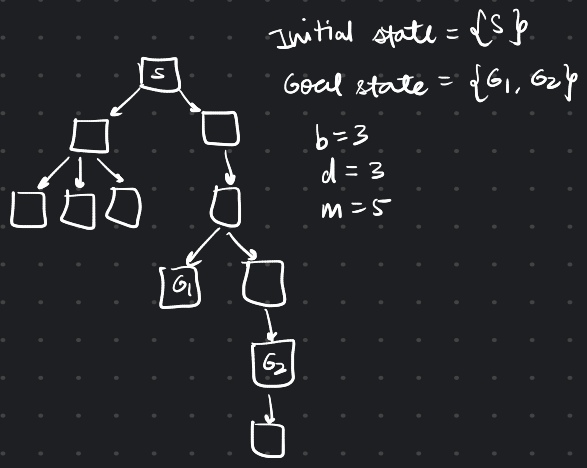
Actions : {L, R, U, D}
(for the blank space)

Measuring problem solving performance

1. Completeness:
   Is the algorithm guaranteed to find a solution, if there is any, and correctly report failures when there isn't one?
2. Cost optimality:
   Are we able to find a solution with the lowest path cost among all solutions?
3. Time complexity:
   How long does it take to find a solution in the worst case?
4. Space complexity:
   How much memory does it take to perform the search?

Parameters

- Branching factor (b):
  Maximum # of successors of any state.
- Shallowest goal depth (d):
  Least possible depth of a goal state from the start state.
- Maximum depth of state space (m):
  Maximum possible depth among all possible states

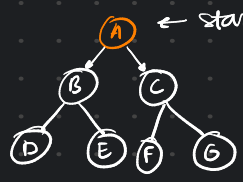


Uninformed search

These algorithms have no clue about how close a state is to the goal.
E.g. BFS, DFS, uniform-cost, depth-limited, depth-first iterative deepening, and bidirectional search.

Breadth-first search

Expand shallowest unexpanded node.
Uses a queue and a "visited" list.



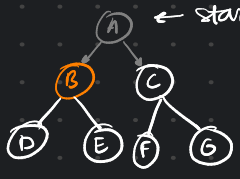
← start node

visited :

A	B	C	D	E	F	G
0	0	0	0	0	0	0

Queue

A



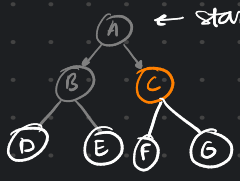
← start node

visited :

A	B	C	D	E	F	G
1	0	0	0	0	0	0

Queue

~~A~~	B	C



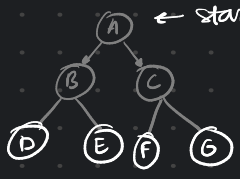
← start node

visited :

A	B	C	D	E	F	G
1	1	0	0	0	0	0

Queue

~~B~~	C	D	E



← start node

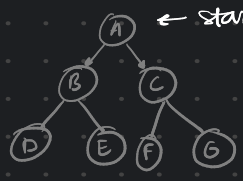
visited :

A	B	C	D	E	F	G
1	1	1	0	0	0	0

Queue

~~C~~	D	E	F	G

and so on...



← start node

visited :

A	B	C	D	E	F	G
1	1	1	1	1	1	1

Queue

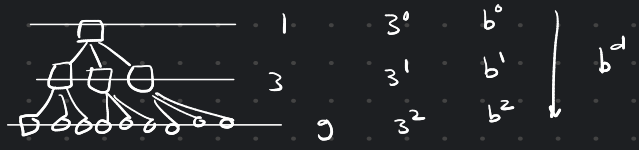
Algo

```
1 Take q as an empty queue.
2 q.enqueue(s).
3 Mark s as visited.
4 while (q is not empty)
5     t = q.dequeue()
6     for all neighbors t' of t
7         if (t' == g) return t'
8         if (t' is not visited)
9             q.enqueue(t')
10            mark t' as visited
```

q ← queue
s ← starting node
g ← goal state

Complete? Yes.
It'll definitely reach the goal (provided that b is finite).

Time? $O(b^d)$



we'll stop at the shallowest depth at which we reach the first goal.

time = $b^0 + b^1 + \dots + b^d = O(b^d)$

Space? $O(b^d)$

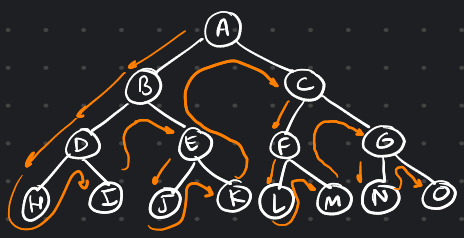


← we'll store last level, b^d nodes.

Optimal? Yes, if step-cost is 1. (equal edge weights)

Depth-first search

Expand the deepest unexpanded node.



A stack is used.

```
take S as an empty stack.
S.push(start)
mark start as visited.
while (S is not empty):
    t = S.pop()
    if (t == goal) return t
    else:
        for t' in neighbors(t):
            if t' is not visited:
                S.push(t')
                mark t' as visited
```

Complete? No.

If max depth is ∞, DFS will go on searching, never reaching the soln.



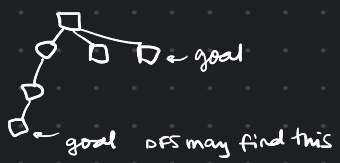
Time? $O(b^m)$

Space? $O(bm)$

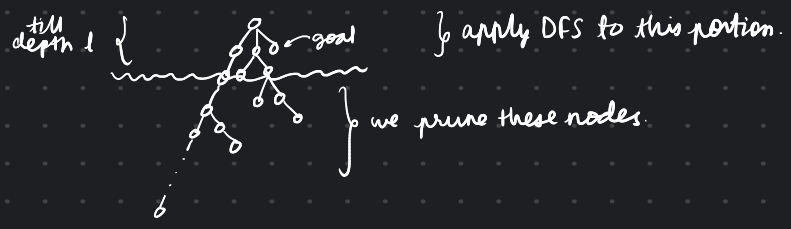


Optimal? No.

also not. It's not even complete



Depth-limited search



We limit our search to some depth l to avoid the infinite depth problem.
We treat the nodes at depth l as if they have no successors.
Problem? If the goal lies beyond l, we may not find it. Finding the optimal l would need some prior knowledge regarding the solution space.

```
1 DEPTH-LIMITED SEARCH
2 take S as an empty stack.
3 S.push(start)
4 mark start as visited.
5 while (stack S):
6     t = S.pop()
7     if (t == goal) return t;
8     if (depth(t) > l) continue;   <- depth-limiting
9     for t' in neighbors(t):
10        if t' is not visited:
11            S.push(t')
12            mark t' as visited.
```

Complete? No. If $d > l$, not complete.

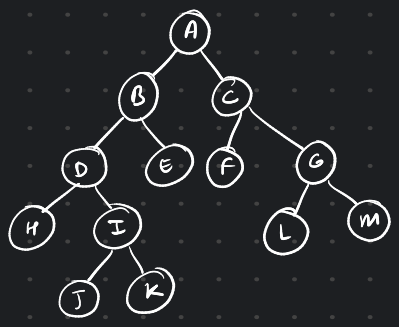
Time? $O(b^l)$

Space? $O(bl)$

Optimal? No.

Iterative deepening depth-first search

```
1 ITERATIVE-DEEPENING DFS
2 for depth d' = 0 to ∞:
3     r = DEPTH_LIMITED_SEARCH(start, d')
4     if (r == goal):
5         return r
```



1st iteration : depth = 0 — A

2nd iteration : depth = 1 — ABC

3rd iteration : depth = 2 — ABDECFG

4th iteration : depth = 3 — ABDHIECFGLM

5th iteration : depth = 4 — ABDHIJKECFGLM



Complete? Yes.

Time? $O(b^d)$

Space? $O(bd)$

Optimal? Yes. If step count is 1.

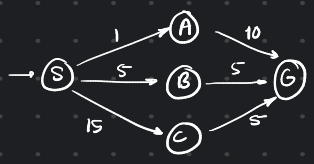
$(d+1)\,b^0 + (d)\,b^1 + (d-1)\,b^2 + \cdots + 1 \cdot b^d = O(b^d)$

(b^0: # of nodes at level 1; b^1: # of nodes at level 2)

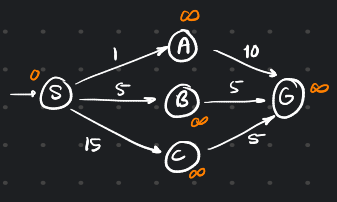
Uniform-cost search

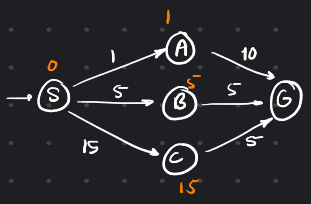
- Dijkstra's shortest path algorithm.
- Data structure: priority queue.

> Dijkstra's algorithm, which is perhaps better-known, can be regarded as a variant of uniform-cost search, where there is no goal state and processing continues until all nodes have been removed from the priority queue, i.e. until shortest paths to all nodes (not just a goal node) have been determined

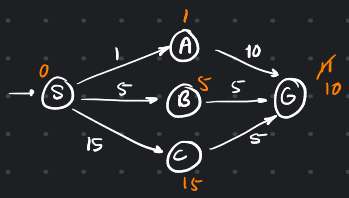


Dijkstra's works by relaxation.



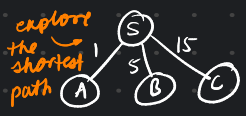


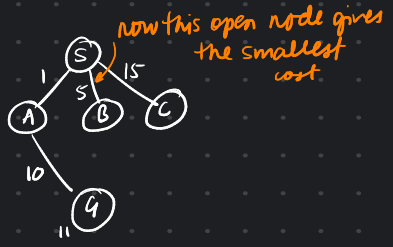
(A,1), (B,5), (C,15)

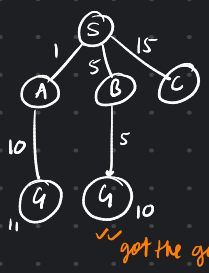


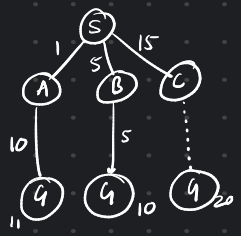
A → 1
B → 5
C → 15
G → 10

As a tree:







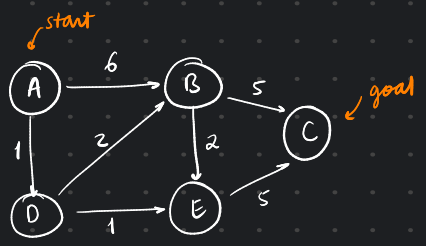



```
1 UNIFORM-COST SEARCH
2 q = priority_queue()
3 q.enqueue(start, 0)
4 while q is not empty:
5     t = q.dequeue()
6     if t is a goal state:
7         return t
8     else:
9         for each t' in expand(t, cost_function):
10            if (t' is not visited) or (t'.cost < visited[t'].cost):
11                visited.append(t', t'.cost)
12                q.enqueue(t')
```

Expand function:

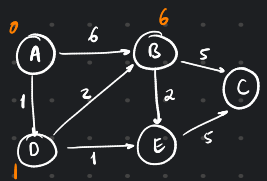
```
1 function expand(node n, cost_function):
2     nodes = []
3     for n' in neighbors(n):
4         n'.cost = n.cost + cost_function(n, n')
5         nodes.append(n')
6     return nodes
7
```

Example:



	A	B	C	D	E
visited	0	x	x	x	x

priority queue: (A,0)

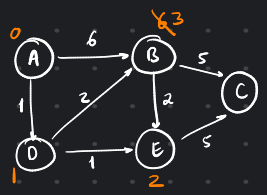


	A	B	C	D	E
visited	0	6	x	1	x

priority queue: ~~(A,0)~~

t = (A,0)

expand(A) → {(B,6),(D,1)}

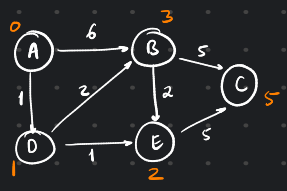


	A	B	C	D	E
visited	0	6	×	1	×
		3			2

priority queue: ~~(D,1)~~ (B,6)

t = (D,1)

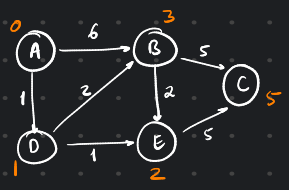
expand (D) → {(E,2),(B,3)}



	A	B	C	D	E
visited	0	3	×	1	2
			3		

priority queue: ~~(E,2)~~ (B,3)

expand (E,2) → {C,5}

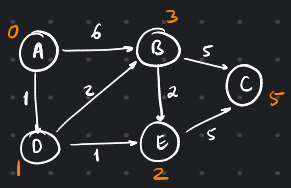


	A	B	C	D	E
visited	0	3	5	1	2

priority queue: ~~(B,3)~~ (C,5)

expand (B) → {(E,5),(C,8)}

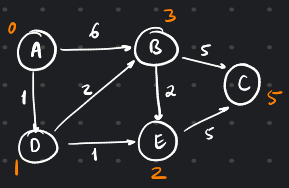
update neither



	A	B	C	D	E
visited	0	3	5	1	2

priority queue: ~~(C,5)~~

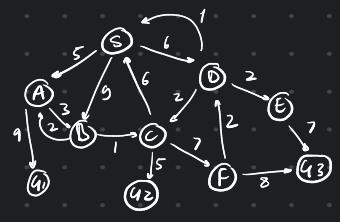
expand (C) → ϕ



	A	B	C	D	E
visited	0	3	5	1	2

priority queue: ( )

Example:



S	A	B	C	D	E	F	G_1	G_2	G_3
0	∞	∞	∞	∞	∞	∞	∞	∞	∞



S	A	B	C	D	E	F	G_1	G_2	G_3
0	~~∞~~ 5	~~∞~~ 9	∞	~~∞~~ 6	∞	∞	∞	∞	∞

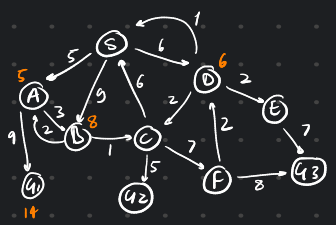
~~(S,0)~~ (A,5) (B,9) (D,6)



S	A	B	C	D	E	F	G_1	G_2	G_3
0	~~5~~ 5	~~9~~ 9	∞	6	∞	∞	~~∞~~ 14	∞	∞

~~(A,5)~~ (B,9) (D,6)

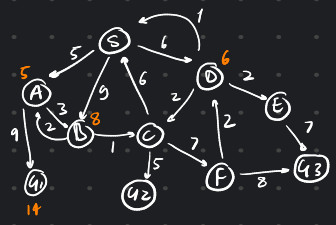
(A,5) → (B,8), (4, 14)



S	A	B	C	D	E	F	G_1	G_2	G_3
0	5	8	∞	6	~~8~~	~~∞~~	14	∞	∞

~~(D,6)~~ (B,8) (G_1,14)

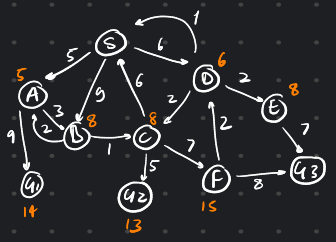
(D,6) → {(C,8), (S,7), (E,8)}
× × update



S	A	B	C	D	E	F	G_1	G_2	G_3
0	5	8	~~8~~	6	8	~~∞~~	14	∞	∞

~~(C,8)~~ (E,8) (B,8) (G_1,14)

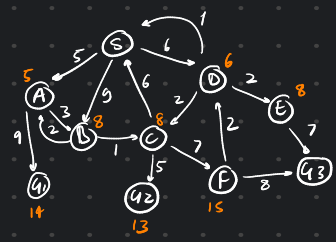
(D,6) → {(C,8), (S,7), (E,8)}
update × update



S	A	B	C	D	E	F	G_1	G_2	G_3
0	5	8	8	6	8	~~14~~ 13	14	~~∞~~ 13	∞

~~(C,8)~~ (E,8) (B,8) (G_1,14) (F,15) (G_2,13)

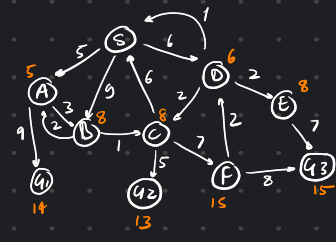
(C,8) → (S,14), (F,15), (G_2,13)
× update update



S	A	B	C	D	E	F	G_1	G_2	G_3
0	5	8	8	6	8	15	14	13	~~15~~

~~(E,8)~~ (B,8) (G_1,14) (F,15) (G_2,13) (G_3,15)

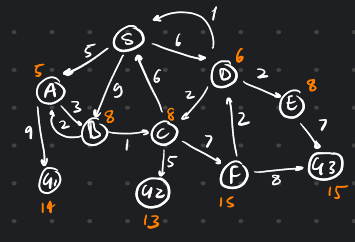
(E,8) → {(G_3, 15)}



S	A	B	C	D	E	F	G_1	G_2	G_3
0	5	8	8	6	8	15	14	13	15

~~(B,8)~~ (G_1,14) (F,15) (G_2,13) (G_3,15)

(B,8) → {(C,9) (A,11)}
× ×



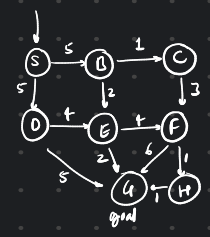
S	A	B	C	D	E	F	G_1	G_2	G_3
0	5	8	8	6	8	15	14	13	15

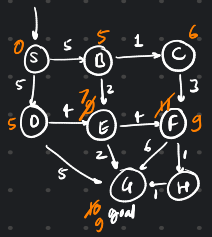
(G_1,14) (F,15) ~~(G_2,13)~~ (G_3,15)

on dequeueing a goal, return it.

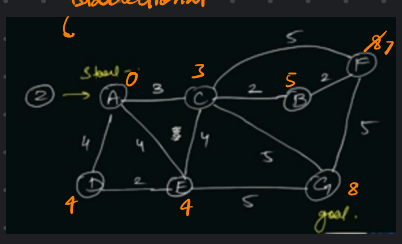
(G_2, 13)

H.W.





Bidirectional



Complete? Yes. we assume that there are no negative edges.

Time? $O(b^d)$ $O(b^{1+\lfloor \frac{c}{\epsilon} \rfloor})$

total optimal cost = C
minimum cost edge = ε

$d = 1 + \lfloor \frac{c}{\varepsilon} \rfloor$



(every edge costs at least ε)

$c = d\varepsilon$

goal is at depth d

Space? $O(b^d) = O(b^{1+\lfloor \frac{c}{\epsilon} \rfloor})$

Optimal? Yes. If we don't consider -ve cycles.

24

If the branching factor is b, every time you expand out a node, you will encounter k more nodes. Therefore, there are

- 1 node at level 0,
- b nodes at level 1,
- b^2 nodes at level 2,
- b^3 nodes at level 3,
- ...
- b^k nodes at level k.

So let's suppose that the search stops after you reach level k. When this happens, the total number of nodes you'll have visited will be

> $1 + b + b^2 + ... + b^k = (b^{k+1} - 1) / (b - 1)$

That equality follows from the sum of a geometric series. It happens to be the case that $b^{k+1} / (b - 1) = O(b^k)$, so if your goal node is in layer k, then you have to expand out $O(b^k)$ total nodes to find the one you want.

If C is your destination cost and each step gets you ε closer to the goal, the number of steps you need to take is given by C / ε + 1. The reason for the +1 is that you start at distance 0 and end at C / ε, so you take steps at distances

> 0, ε, 2ε, 3ε, ..., (C / ε)ε

And there are 1 + C / ε total steps here. Therefore, there are 1 + C / ε layers, and so the total number of states you need to expand is $O(b^{(1 + C / \varepsilon)})$.

Hope this helps!

7

templatetypedef's answer is somewhat incorrect. The +1 has nothing to do with the fact that the starting depth is 0. If every step cost is at least ε > 0, and the cost of optimal solution is C, then the maximum depth of the optimal solution occurs at floor(C / ε). But the worst case time/space complexity is in fact $O(b^{(1+\text{floor}(C/\varepsilon))})$. The +1 arises because in UCS, we only check whether a node is a goal when we select it for expansion, and not when we generate it (this is to ensure optimality). So in the worst case, we could potentially generate the entire level of nodes that comes after the goal node's residing level (this explains the +1). In comparison, BFS applies the goal test when nodes are generated, so there is no corresponding +1 factor. This is a very important point that he missed.

0

To be fair I struggle too, to see a reason for it to be included.

I'll try to argumentate: assume that the optimal path has length N and therefore you have costs $c_1, \ldots, c_N$ all positive and bigger than some $\epsilon > 0$. Such ϵ can't be bigger than the minimum between $c_1, \ldots, c_N$.

Now, to need that $+1$ correction factor it should be possible that

$$\frac{c_1 + \cdots + c_N}{\epsilon} < N$$

Since $\epsilon < min\{c_1, \ldots, c_N\}$ I could try to prove instead that

$$\frac{c_1 + \cdots + c_N}{min\{c_1, \ldots, c_N\}} < N$$

But you can divide both members by N and multiply by $min\{c_1, \ldots, c_N\}$ obtaining that

$$\frac{c_1 + \cdots + c_N}{N} < min\{c_1, \ldots, c_N\}$$

On the left hand side you have the average of the costs, which of course can't be smaller than the minimum of the costs, therefore that has no solution. This means that even without the +1 factor you can't underestimate the length of the path to the optimal solution using $\lfloor C^*/\epsilon \rfloor$.

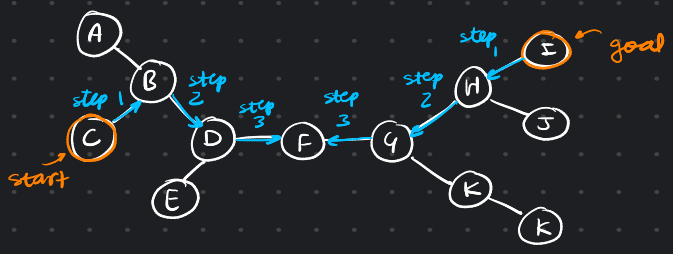
On the flip coin by the way when you compute that $\lfloor C^*/\epsilon \rfloor$ you're actually trying to find an upper bound for the length of the optimal path, so if you add +1 (equivalent to compute the ceiling instead of floor) you still get an upper bound. Also, since you're computing an $O(\circ)$ keep in mind that if a function is an $O(b^k)$ then it's an $O(b^{k+c})$ for every $c > 0$ so although not necessary the book isn't technically wrong.

EDIT: Take all I said with a grain of salt, I'm a student, too and still have to take the AI exam

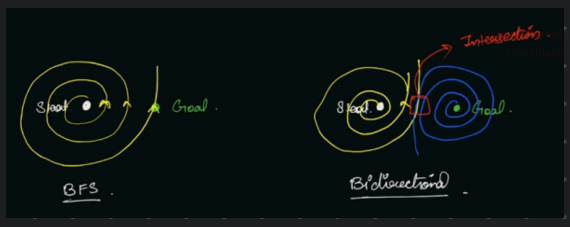
EDIT 2: I thought that another possible reason why you would expand (at most) b more nodes after reaching the depth of the optimal goal is that you perform the goal test the moment you extract the node from the frontier and not before insertion. This is necessary cause it grants us that when we extract a node from the frontier we reached it through the path with lowest cost from the initial state, but it will cause us to expand at most one more level before realizing a node in the frontier is indeed a goal.

Bidirectional search

- Start searching from the start node (forward search). Parallely, start searching from the goal state (backward search).
- Hope that both the searches meet somewhere in the middle, giving an optimal path.



The type of search is chosen as wanted / reqd.




```
BIDIRECTIONAL_SEARCH
take q1, q2 as the empty queues.
q1.enqueue(start)
mark start as visited.
q2.enqueue(goal)
mark goal as visited.
while (q1 is not empty) and (q2 is not empty):
    t1 = q1.dequeue()
    t2 = q2.dequeue()
    if (q1 is not empty):
        for t1' in neighbors of t1:
            if (t1' == s or t1' == q1):
                return t1'
            if (t1' is not visited):
                q1.enqueue(t1')
                mark t1' as visited.
    if (q2 is not empty):
        for t2' in neighbors(t2):
            if (t2' == s or t2' == q1):
                return t2'
            if (t2' is not visited):
                q2.enqueue(t2')
                mark t2' as visited.
```

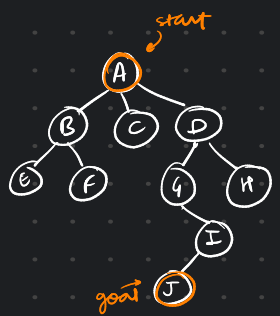
2 separate visited lists are NOT reqd. when BFS is used.

Complete? Yes.

Time? $O(b^{\frac{d}{2}})$ $O(b^{d/2} + b^{d/2}) = O(2b^{d/2}) = O(b^{d/2})$

Space? $O(b^{\frac{d}{2}})$

Optimal? Yes. (BFS is optimal)

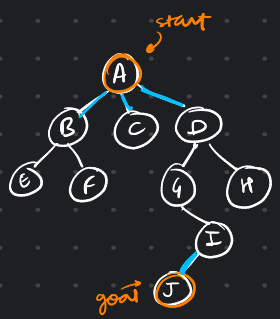


$q_1 =$ A

$q_2 =$ J

visited

A	B	C	D	E	F	G	H	I	J
x	x	x	x	x	x	x	x	x	

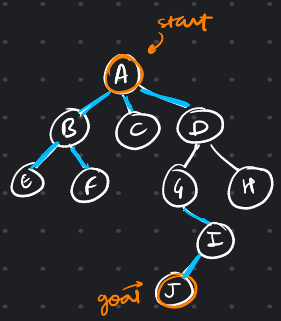


$q_1 =$ ~~A~~ BCD

$q_2 =$ ~~J~~ I

visited

A	B	C	D	E	F	G	H	I	J
				x	x	x	x		

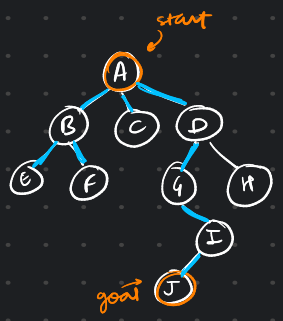


$q_1 =$ ~~B~~CD EF

$q_2 =$ ~~I~~ G

visited

A	B	C	D	E	F	G	H	I	J
							x		

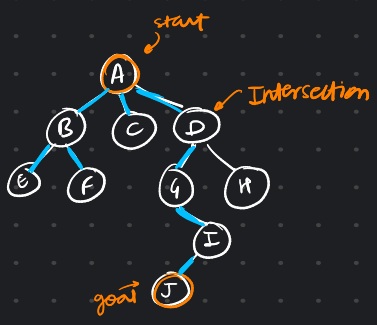


$q_1 =$ ~~C~~D EF

$q_2 =$ ~~G~~ D

visited

A	B	C	D	E	F	G	H	I	J
							x		



Intersection

$q_1 =$ ~~D~~ EF

$q_2 =$ D

visited

A	B	C	D	E	F	G	H	I	J
							x		

